

Faculty of Electrical Engineering, Mathematics and Computer Science

TCS Design Project

Design report

Smart Grid Demonstrator

Group 7:

Egbert Dijkstra	s1700618
Simona Filipović	s1839780
Koen van Hove	s1939777
Pieter Staal	s1776894
Thomas Vrielink	s1599399

Supervisor:

dr. ir. Gerwin Hoogsteen

Index

Index	1
Abstract	3
Introduction	4
Requirements	5
Web-based visualiser	5
DEMKit	5
Interface	7
Global design	8
Design: DEMKit extensions	10
State pusher	10
State structure	10
Entity naming convention	11
API	11
Dynamic model updating	14
Design: Web-based Visualiser	15
Web server	15
Socket.IO events	15
Frontend	17
Design choices	20
Recreating Tactile Triana for the browser	20
Endpoint location	20
Static network configuration	20
Pushing the simulation state	20
Controlling the simulation time	21
Have the Node.JS server in between	21
Dumb/Smart button	21
Testing	22
Unit tests	22
Selenium tests	22
User testing	22
Set-up	23

Design Report - Smart Grid Demonstrator

Evaluation	24
Planning	24
Responsibilities	24
Team evaluation	24
Final result	25
Conclusion	25
Sources	26

Abstract

In order to demonstrate how energy grids work, the Computer Architecture for Embedded Systems group would like to see their energy grid simulation tool DEMKit linked to an energy grid visualisation tool called Tactile Triana. The task of this project was to create that link.

Since Tactile Triana was outdated, the scope of the project changed to building a new web-based visualisation tool, which reuses the assets of Tactile Triana, but otherwise has more capabilities. This visualiser has been designed to be linked to DEMKit. Aside from creating the link and the new visualiser, DEMKit itself has also been extended to allow changes to the central model and simulation state. This way, changes can be made on the visualiser, which will be issued through the link, and then changed in the DEMKit simulation.

The result is a flexible web-based visualisation tool that creates an intuitive visualisation for the simulation provided by DEMKit, that can automatically adapt to new algorithmic changes in DEMKit. Additionally, DEMKit has been modified to allow simultaneous control with multiple visualisers.

Introduction

We are in the middle of an energy transition. A transition from fossil fuels to green energy. However, energy usage is, unlike water or other utilities, difficult to grasp. To provide an insight into energy patterns, several visualisation interfaces were created. Due to our increased energy usage (caused by for example electric cars), and fluctuating nature of wind and solar energy, keeping the energy grid balanced becomes an increasingly difficult challenge.

To help solve this balancing issue, the Computer Architecture and Embedded Systems (CAES) group at the University of Twente has been developing optimisation and control systems for a so-called *smart grid*, where the grid controls how much energy each device gets. For this Decentralised Energy Management, they created a simulator called DEMKit.

Tactile Triana was a previous design project which aids in visualising a power grid of six houses and what happens when devices like solar panels and electric vehicles are added. Tactile Triana can be controlled by an external controller via an API to a certain extent. It will still run its own simulation, but the outside controller can change the planning of devices in houses.

Tactile Triana currently is a completely standalone program. Since it was a design project there has not been any development for the past years and one of the libraries got out of date. In this time a new simulator, DEMKit, was developed with far more features than the Tactile Triana internal simulator had.

A large difference between Tactile Triana and DEMKit is that in Tactile Triana users can add and remove devices dynamically while the simulation is running, while in DEMKit you start with a set configuration and complete the simulation without changing the setup.

Tactile Triana runs its own simulation. For DEMKit to make the planning we want to use with Tactile Triana, DEMKit also has to run a simulation. These simulations obviously will not agree, since the DEMKit one is far more sophisticated and they are completely different implementations. That is why we have decided to recreate Tactile Triana for the browser, called the Web-based Visualiser. This uses DEMKit as a simulation back-end and is capable of making changes to the configuration of the simulation.

Requirements

To determine the capabilities this system should have, we have taken a look at the capabilities of DEMKit and of Tactile Triana. We have also looked at different visualisation tools, such as a SmartHouse that is already linked to DEMKit, to see what is possible. Based on these, and on discussions with the client, we have made a list of requirements, divided into must-haves and could-haves for both the web-based visualiser and the DEMKit extension:

Web-based visualiser

Must have

- It should work with the CAES touchscreen
- It should work on computers
- It should give visual cues for the seasons and the time
- It should be able to make configuration changes to DEMKit
- It should be able to pause and play the simulation
- It should be able to play the simulation at a slow and fast speed
- It should have intuitive touch screen controls
- It should be able to change the settings of devices and push these to DEMKit
- It should have a configuration page where you can set the DEMKit address

Could have

- A balanced configuration for gamification
- Dynamically add houses
- People can join the session on their phone

DEMKit

Must have

- It should be able to push its state to clients
- Clients should be able to change the configuration of DEMKit during the simulation
- Clients can advance the simulation with a /tick command
- It should accept new clients through an endpoint
- Clients should be able to pause the simulation
- When multiple clients are connected, there should be an option to have only 1 master

Could have

- Dynamically add houses
- Change the network configuration

The must-haves have been realized, and their implementation will be discussed in this report. Most could-haves are rather large in scope and have thus not been implemented. We would

need to build an entirely new visualiser for smartphone support, for example, which was not feasible within the time frame of this project. The could-haves that were relatively small in scope, such as balancing the configuration, have been implemented. Also, at the start of the project, when we were still thinking of possibly adding a smartphone version, we started working on a system where multiple clients could connect to DEMKit. While this is possible, it has not been extensively tested. When the idea of smartphone support was abandoned, this became an inessential part of the project, and we decided to focus on getting the system functioning on the touchpad, thus we focused on this with testing.

While we did abandon actual development of a visualiser for smartphones, it is still possible to add this in the future since we took the option of this feature into consideration during development. Our state pusher allows multiple clients and in the API there is an endpoint to add clients to the state pusher. Any future visualiser, including a visualiser for mobile devices, can use this endpoint to register themselves and receive the state.

Interface

Considering that the demonstrator is to be run on a large touch screen, it is paramount that the interface is clear and easy to use. Users should be able to use the smart grid without any explanation. All the grid elements must be clearly displayed. The fact that users can stand on all sides of the screen should be taken into account. This means that all text must be directed towards the user. Users must be able to modify the simulation by dragging devices into the houses, thus connecting them to the network. To make sure the screen is not overflowing with unused devices they will be automatically removed, thus lessening the clutter of the visualisation. Users must also be able to set a number of parameters per device through the settings panel. It is important that the demonstrator is intuitive enough for users to handle the application without a manual or a readme file.

Certain visual cues also exist in order to ease the use of the grid demonstrator. These include:

- Interface adopts a gray hue when the simulation is paused to show inactivity
- When charging a progress bar will be displayed for the devices to indicate the current state of charge
- If a cable is broken it will become orange, once fixed it becomes gray again
- To indicate the flow of the current bolts will move through the cable in the desired direction
- Device power usage and production is shown through colour, the more red it gets the more power it uses, and more green it gets the more power it produces
- If the smart button of the house is selected the button turns green (otherwise it is gray to indicate it is not selected)
- The nodes vary in colour depending if they are going over or under the threshold
- A switch from day to night as well as passing of the seasons are used as indicators of the passage of time



Smart grid demonstrator interface

Global design

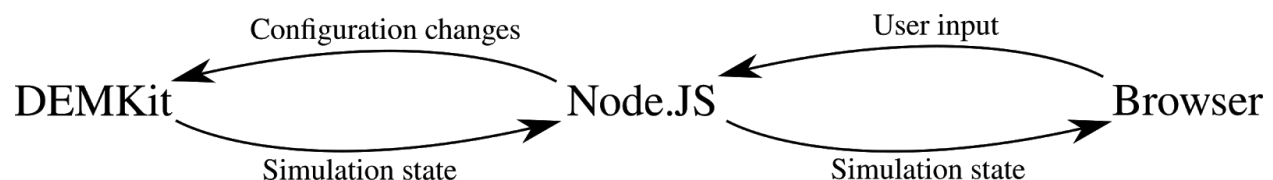
The total system consists of two main components: DEMKit and the Web-based Visualiser. In this section, we will detail the information flow between DEMKit and our Web-based Visualiser since the Web-based Visualiser needs the simulation information to display it to the user, but DEMKit also needs data from the Web-based Visualiser in the form of commands to change those in the simulation. Thus, DEMKit is the model, while the visualiser is the view and the controller in the MVC-model.

To get DEMKit to communicate its internal simulation state to anything we added the 'state pusher'. This is a special DEMKit Host (a host takes care of the simulation) which keeps a list of clients and sends its simulation state on every tick and every change to the simulation, e.g. when a device is added to the simulation.

We also created an API in DEMKit that exposes endpoints to allow those to be controlled from the outside. This API accepts commands that drive the simulation and change its configuration. It is, for example, possible to add and remove arbitrary devices through this API.

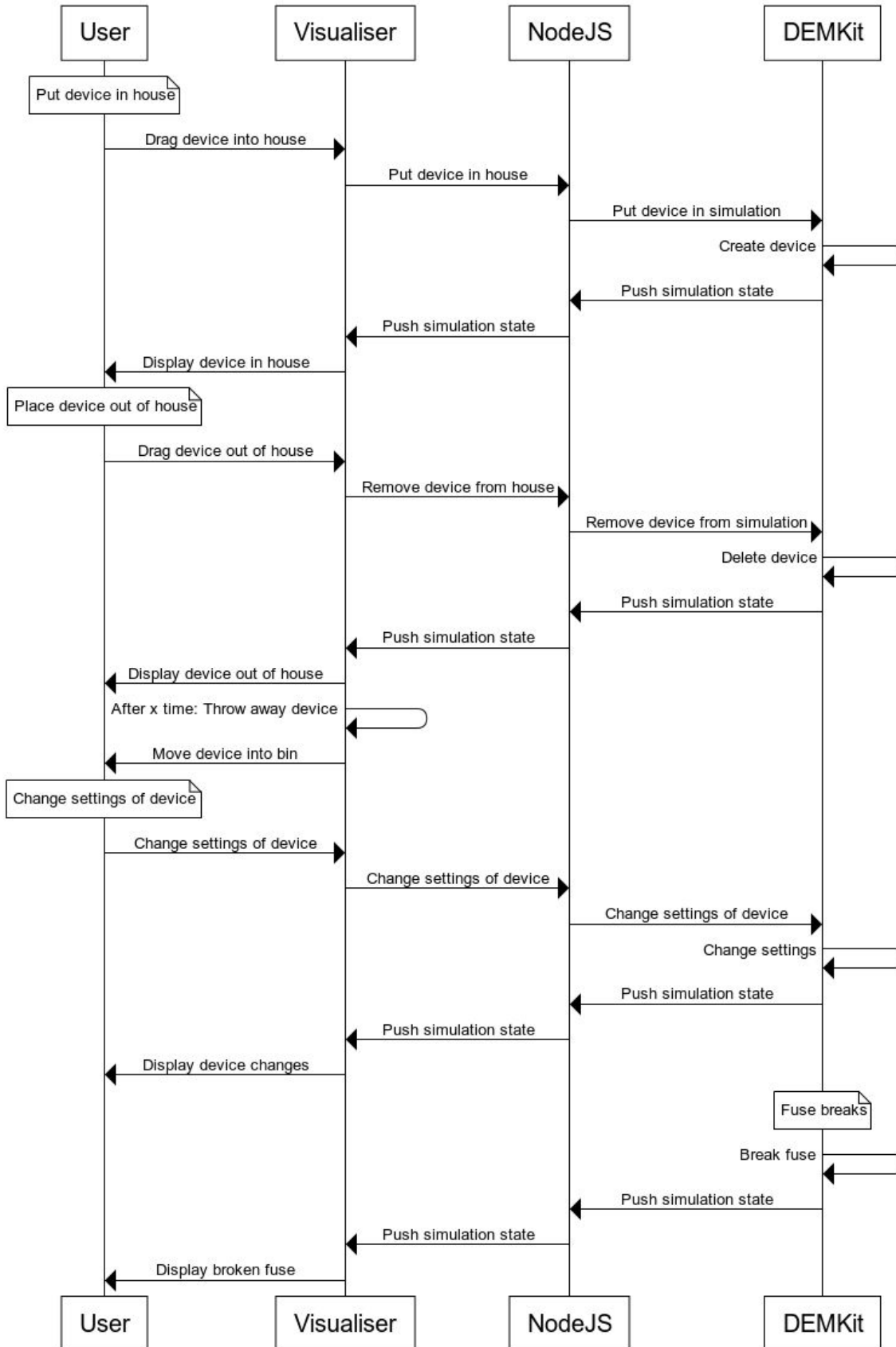
The Web-based Visualiser is run by starting the Node.JS server for it. It can then be viewed in any browser, but Firefox is recommended for its behaviour on touch screens. This means that the Web-based Visualiser itself consists of two components: the frontend and the backend. These components communicate through Socket.IO. With Socket.IO events can be sent to and from the server with data payloads. The events coming from the frontend to the server are used to signify user actions like adding a device to a house or changing a setting of a device. The server then translates these actions into DEMKit compatible API calls to modify the internal models. Events from Node.JS to the frontend are mostly for passing along data received from DEMKit, like the simulation state. All these events are detailed in the section on the design of the Web-based Visualiser.

Below a diagram of the described data flow within our total system.



And on the next page the flow of data can be seen in effect. The sequence diagram shows three actions the user can take to affect the system and a situation that can occur without user input.

Design Report - Smart Grid Demonstrator



Design: DEMKit extensions

To get the necessary data out of DEMKit we added two main components:

1. `/components/hosts/pushHost.py`: This is the special host required for our application, explained in the 'state pusher' subsection.
2. `/components/api/designApi.py`: Here is all the overhead for and the endpoints of the API. The total API specification is in this section under the subsection 'API'.

Some endpoints or principles used by endpoints are also detailed here in their own subsection.

State pusher

To inform connected clients about the internal simulation state we created a new host based on `simHost` and `restHost` in `/components/hosts/pushHost.py`. This `PushHost` initializes our `DesignApi` component. Since the `DesignApi` has a reference to the used host, it then waits for inputs like `/tick` from visualisers. The `DesignApi` then calls the corresponding functions on the `pushHost`, updates its time administration variables, and calls `PushHost`'s `push` function. This `push` function goes through the `PushHost`'s list of clients.

These clients only have to follow one simple pattern: they have to be a REST API with at least the endpoint `/dem/state`. With this in place, the `PushHost` only has to know the address of all the clients and it can push to it. The client is then free to visualise the data in any way it sees fit.

For some endpoints in the API, we also push the state. This happens when a device is added or removed from the simulation. This is done to make sure that any connected client immediately knows when there is a change in the simulation.

State structure

To visualise the network we need four basic DEMKit data types, controllers, devices, environment, host, and the network. In the state pusher these lists are filled based on the current simulation state.

```
{
  controllers: [{}]
  devices: [{}]
  environment: [{}]
  host: [{}]
  network: {
    nodes: [{}]
    edges: [{}]
```

```
}  
}
```

Entity naming convention

DEMKit allows any string to be the name of a device, as long as it is unique. This name is then used for looking up devices when e.g. removing them. This means we were free to choose a naming scheme. Important to note is that these names contain information that is necessary in both the front-end and back-end or in only one of the two places. This scheme looks as follows for devices.

```
[devicetype]-[identifier]-House-[ housenumber]
```

This name consists of three components.

1. `devicetype`: one of the following: SolarPanel, DishWasher, Boiler, Battery, WashingMachine, or Vehicle. We use this property to determine for example what sprite to draw in the front-end.
2. `identifier`: a random number between 0 and 4294967295 (0xffffffff in hexadecimal) represented as a string of hexadecimal characters. This is used in the front-end to distinguish multiple devices of the same device type from each other. Keeping track of this in DEMKit as well allows us to work with multiple frontends running on the same DEMKit instance while knowing which device is different from others.
3. `housenumber`: an integer between 0 and 5, signifying which of the six houses this device belongs to. Apart from the electricity flow simulation, DEMKit does not have a notion of 'houses', just devices grouped under house controllers. We added this property to easily see which house in our visualiser a device belongs to.

For controllers, we simply add the string `SmartHouse` before the name of the device it controls. This means controllers for a smart device will have a name that looks as follows:

```
SmartHouse[devicetype]-[identifier]-House-[ housenumber]
```

API

Since the visualiser needs to update models in DEMKit we decided to create our own API, in `/components/api/designApi.py`. Like the existing `eveApi`, we decided to keep this API generic. It is possible to add any class to the entities list and execute functions on them. This could be a security risk but as we discussed with the client this is not a priority since this software is mostly used in academic situations and for closed groups.

For DEMKit to work together nicely with an outside visualiser we realized it is necessary for the outside visualiser to be able to 'run' DEMKit. This means DEMKit will not simulate on its own but

instead will wait until the visualiser sends a ‘tick’ command. That is why the DesignApi has two tick endpoints. One takes an argument in the link for the amount of ticks, and the other simply performs a single tick.

API overview

Most of these endpoints are used not to obtain information from DEMKit, but to control it. This means that the output that is given by endpoints is usually less important than the side effects. After the overview we will detail exactly what the side effects are for each endpoint.

Endpoint	Input	Output
/tick	-	{ time: <current sim time>, isMaster <bool> }
/tick/<n>	-	{ time: <current sim time>, isMaster <bool> }
/remove	{ entity: <name> }	{ success: <bool> }
/addClient	Nothing or { port: <port> }	{ success: <bool> }
/addEntity/generic	Entity class and properties	{ success: <bool> }
/editEntity	{ name: <name>, properties: {<property>} }	{ success: <bool> }
/makeSmart	{ devices: [<name>] }	{ success: <bool> }
/makeDumb	{ devices: [<name>] }	{ success: <bool> }
/repairCable	{ name: <name> }	{ success: <bool> }
/control/request	{ clientId: <id> }	{ success: <bool> }
/control/release	{ clientId: <id> }	{ success: <bool> } or { error: <error> }

In this overview we used terms enclosed in angle brackets to signify certain objects. We will go into detail of what exactly those objects must contain to be interpreted correctly by the API.

<bool>

A simple boolean as defined in the JSON standard, so either the string “true” or the string “false”.

<port>

An integer between 0 and 65536. The API does not check if the value satisfies this condition and will behave unexpectedly when another value is supplied.

<id>

A client ID. This can be any string. In our use-case the Socket.IO socket id is used as the master ID as it can be used to uniquely identify clients.

<name>

The name of an entity in the DEMKit simulation. DEMKit allows any name to be set, but we use a custom naming convention as detailed in a previous subsection.

<error>

A string detailing the exact exception python threw.

<property>

This can be any key-value pair of a property of the entity with name <name>. If the type of the variable the property is trying to set is an integer or boolean the supplied value will be cast to that type. In the case of the boolean variable the strings “on”, “true”, “True” are interpreted as True.

Below we will list some side effects of invoking an API method.

/tick and /tick/<n>

Advances the simulation by respectively 1 or n ticks. In our configuration, this means that the simulation will advance by fifteen minutes.

/remove

Removes the entity with name <name> from the simulation. Success if true if the entity was successfully removed.

/addClient

Adds a client to the state pusher with the requester’s IP address, and port number that is specified (or 3002 if not included). Returns success true if executed correctly, or false if the client was already added or could not be added.

/addEntity/generic

Adds the specified entity to the simulation. The endpoint is structured in a way that makes it very flexible regarding the entity type. It expects the class (including path) of the entity to be added, the entity type (one of the seven) and the entity structure. The entity structure consists of parameters, properties and functions. The class is instantiated with a list of arguments provided as parameters. Then the properties of the class are set using the key-value dictionary provided in properties, and a list of functions belonging to the new class are executed (a list of objects containing function names and the parameters). Returns success true if executed correctly, or false if something went wrong.

/makeSmart

This endpoint converts any list of devices into ‘smart’ devices. We use it to convert all devices in a house to smart devices. DEMKit does not have a notion of houses so the caller of this endpoint, in our case the Node.JS server, has to group devices per house and send them to this endpoint. The endpoint expects a list of devices and creates smart controllers for them. Always returns success true, ignores devices that cannot be found.

/makeDumb

Opposite of makeSmart. Expects a list of the smart controller names, shuts them down and removes them. Always returns success true, ignores devices that cannot be found.

/repairCable

It calls the reset method of the cable with the supplied name, thereby 'repairing' it.

/control/request

Requests the control of the simulation. If no simulation master is set then the supplied <id> will be set as the new master. Any tick, remove, and add request will check the ID of the requester, if it is not the same as the set master it will not be executed.

/control/release

If this clientId had control, it releases the control of the simulation. Any other client can claim control afterwards with the /control/request endpoint.

Dynamic model updating

The internal DEMkit model discerns seven different types of entities:

Device, Controller, Meter, Flow, Environment, Cost and Component. Each of these entities is added to a list of their own entity type (e.g. Controller to controllers), and to global list entities. Dynamically adding a new entity to the model is done by adding the entity to both lists, and consequently calling the startup function of the entity. This is handled in the addDevice, addController, etc. methods. This will cause the entity to become available to the simulator in the next tick. The simulator then takes care of taking it into account during the simulation and adding it to the database output.

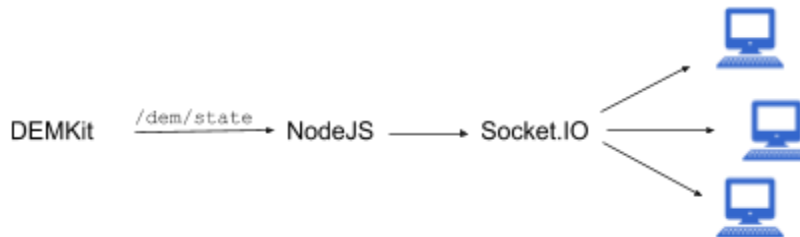
Removing an entity works in exactly the same way. An entity or name of an entity gets passed to removeDevice, removeController, etc., and the entity gets removed from both the entities list and the shutdown function is called. The entity is completely deleted from the simulation, hence the simulator will no longer take it into account during the next tick.

The system also supports smart controllers. When a house is made smart, an auction controller is added to the devices currently in the house. Any new device will then also automatically have a smart controller added to it. The opposite happens for making a house dumb again.

Design: Web-based Visualiser

Web server

Our frontend runs on a NodeJS server. At its core, it handles two things; Serving the HTML file for the user as well as the connection layer between said HTML file and DEMKit. This is achieved by creating an endpoint `/dem/state` on which DEMKit posts its data.



As seen in the figure above DEMKit pushes its state to nodejs. NodeJS uses Socket.IO to further publish it to its connected clients. The logic here is rather dumb as it sees the DEMKit as its master and publishes everything that it sends.

Socket.IO events

To communicate with the DEMKit simulator the frontend needs to be able to make requests to its API. Our frontend does this in two steps. First, it emits an event over Socket.IO to our NodeJS backend. The NodeJS backend then interprets the event, expands the event a little with e.g. a client ID, and then sends it to DEMKit. We do this instead of making calls directly to prevent cross origin requests, and because this way the NodeJS server can be responsible for managing certain default values and generating the exact JSON that DEMKit expects so we do not have to do that in the frontend.. Here we will cover all the Socket.IO events available in the server and client.

Client - data

In this event the client receives the DEMKit state. It updates a global variable called `DEMData` to the new state. This variable stores the received state as a Javascript object or if nothing has been received it is null. If this is the first time that we receive DEMKit data we initialise the network object.

Here we also check whether devices got removed from the simulation or were added, and act accordingly. This way if a device is added by an outside source it will show up immediately in the visualisation.

Client - tickResponse

When the client tries to do a tick, DEMKit will answer whether it was successful. A tick could fail if the client is not a master. If this is the case the client will hide the play and fast forward button, because they cannot be used anyway.

Client - obtainedControl

Emitted when this client has gained control of the simulation. It sets the global isMaster variable to true and it unhides the fast forward and play button.

Server - disconnect

Emitted by the client when it disconnects from the socket. The server will post a release control request to DEMKit. It does this for every client, even if they are not the master because the NodeJS server does not know who the master is and if they were not the master DEMKit will just do nothing.

Server - requestTick

Requests a single tick from DEMKit. The server adds the socket id to the request as the clientId parameter.

Server - requestTicks

Works the same as the requestTick event, but requests five ticks to speed up the simulation.

Server - requestControl

Requests control from DEMKit. If it is successful it emits the obtainedControl event to the client that requested the control.

Server - removeDevice

Takes the name of the entity to be removed from the payload sent from the client and sends that entity name to DEMKit. DEMKit removes it from the simulation.

Server - addDevice

Here the client has very little control of the actual configuration of the device it wants to add to the simulation. The NodeJS server has some preloaded configurations per device type and simply uses those to send the add request to the generic DEMKit addEntity endpoint. The NodeJS server knows what configuration to use exactly and which classes to use for which device.

Server - editDevice

Calls the generic editEntity endpoint to change the properties of entities in the simulation. We use this in the edit menu opened by tapping on a device.

Frontend

The web-based visualiser is an HTML5 canvas element onto which we draw all the necessary components to simulate Tactile Triana. We have organised the code such that there is a `main.js` where we initiate all our global variables and start the game loop. This game loop requests an animation frame from the browser. The browser will allocate frames in such a way that the loop will run about sixty times per second.

In the game loop, we first update all the game objects that need updating. This means, for example, the network visualisation, which shows tiny bolts of electricity in a cable going along at certain speeds and need to be changed every frame.

After all the necessary updates, we go on to the drawing phase. By first drawing the background we erase the previous frame. Afterwards, we draw all game objects in the correct order.

Game objects are organized as their own classes. Here we present an overview

House

The House class is used to represent the houses. In its constructor, we have the possibility to construct it with a given set of devices. If we don't have that available we can also add said devices through the `addDevice( ... )` method.

House also has a function that is used to check if a device is inside the house or not. A device has to be fully inside a house to be counted as inside. This is used to check if a device needs to be added and or removed from said house.

Device

The device class can construct new device game objects based on parameters (with the constructor) and from DEMKit data (with the static `fromDemData` method). Devices have an update method which is called every frame from the game loop. In this method, we calculate a new position based on its velocity. Devices can gain velocity by being dragged by the user.

Devices also have some methods attached to event listeners for mouse movements like a grab and release method. In the event listener we determine which device is the one being clicked with the help of the device's `isClicked` method.

To show device power usage and production we change the color of the device. The device gets more red the more it uses, and more green the more it produces. This scale is relative to the constant `CONSUMPTION_VISUALISATION_ROOF`. This value is currently set to 2000 Watts, this means that when a device consumes 2 kilowatts or more, it will be fully red (`#ff0000`), when the device produces electricity its usage will be negative, so the colour will move towards bright green (`#00ff00`).

Touch events

To move the devices on the screen we needed to grab the users input to see what is happening with the mouse or touch. Javascript has `EventListeners` to enable us to grab the users input. However, there is a difference between mouse input and touch (and gestures with that). To solve this we created functions that can operate with both of said input types, `GrabDevice`, `MoveDevice` and `ReleaseDevice`. With the given `X` and `Y` coordinates we search the device list to see if it intersects with a device. This, however, can form an issue if we have two devices that are stacked. To circumvent this we are searching the device list in reverse order. In this case we always get the device that is on the top side of the stack because we are drawing the array in the regular order.

Device despawning

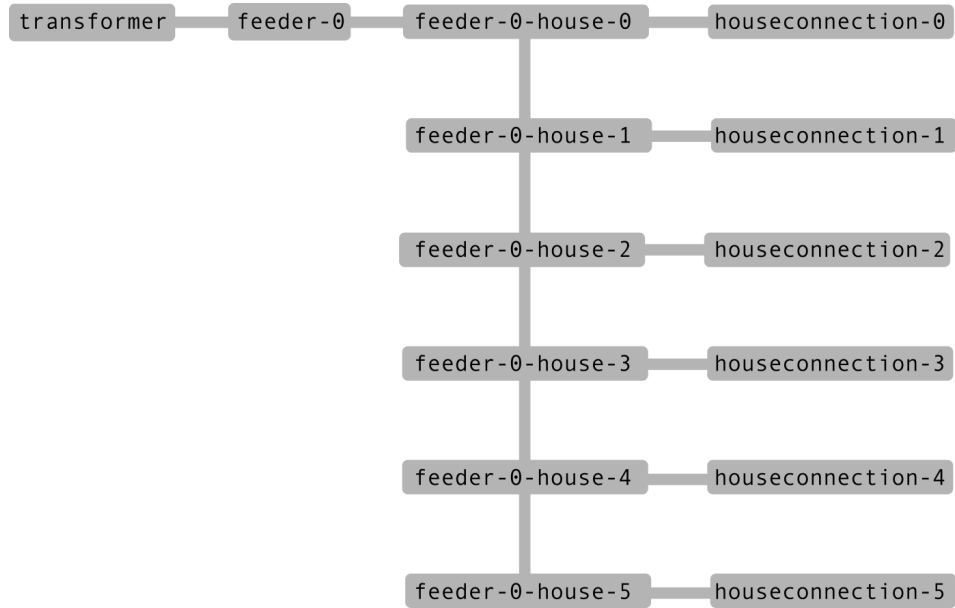
To remove the clutter on the screen we need to despawn devices that are not added to a house. We can do this in two manners; drag the device to a trash can or despawn it. But the question that then arises is when and how? In the `Device` class we have a constant named `ttl` (time to live). This regulates how long a spawned device can live outside of a house. Every update (to see if needs to move or we need to open the settings menu) we then check if the device is in a house and if it exceeded this threshold. Ifso we set the goal position property of the device, this goal position property is normally `None`. The next update we set the device's velocity to be in the direction of the trash can, with a magnitude depending on its distance to the trash can icon. This way the device moves by itself towards the trash in a pleasing manner, very similar to tactile triana.

Network

The network class is used to manage the nodes and edges of the network graph. It can be setup using a network object received from DEMKit. It then creates node and cable objects based on the received data. It also contains the method to be updated from DEMKit data. Every time DEMKit data is received the network is updated with this `setNetwork` function.

All connections and names in the network should be as in the diagram below, because we use the names as identifiers when defining the layout of the network in the game world.

Design Report - Smart Grid Demonstrator



Node

The node does not have an update function, as it is rather simple. It just has a draw function that draws a square at the specified size and colour.

Cable

Cables do have an update method because their `CurrentParticles` have to be updated. The cable's `init` function calculates where to start and end the line to be drawn, and it calculates some direction vectors that the `CurrentParticles` will use.

CurrentParticle

These are the tiny electricity bolts flowing through the cable. They get the direction of movement from their parent cable (calculated in its `init` function). The speed is based on the amount of current flowing through their parent cable.

To ensure the bolts are evenly spaced their movement along the cable actually exceeds the cable length. A `max` attribute is calculated in the constructor, this attribute satisfies the condition that `max % BOLT_DISTANCE == 0`. To prevent bolts showing up outside their cable we added a check in the draw function that ensures bolts are only drawn when their `place` is smaller than the cable length.

Design choices

In this section, we will describe some of the major design decisions we made while developing our solution.

Recreating Tactile Triana for the browser

Since Tactile Triana was developed as a design project in 2015, no work has been done to keep it up to date. While simply running it still works fine, we learned that it uses an old version of java and uses an outdated user interface library. Since learning how a program works requires building a mental model of the program in your mind, it is almost the same effort to build a similar program from scratch and building the mental model along the way [1]. This is why we decided, in agreement with the client, to start from scratch with up to date technologies we are familiar with.

Endpoint location

During development, we added some very specific endpoints to DEMKit for adding certain devices with a pre-existing configuration. This saved us making more complicated calls to the generic addDevice endpoint on the DEMKit API. However, after a discussion with our client we found they would prefer more generic endpoints in the DEMKit API. This was preferred because DEMKit is primarily a tool used for research, and therefore subject to changes which our more specific API then might not longer support. Therefore, it would require work to keep it up to date, without which this project would go the way Tactile Triana did. Thus, we decided to move the endpoints to our Node.JS server and call the generic addDevice endpoint.

Static network configuration

The Web-based Visualiser expects a very specific network configuration, as detailed in the section on its design. In the render section for the network, we solidify this requirement even more by hard-coding the locations of all nodes in the network we expect to receive. We chose to take this approach because our scenario is also very specific. Our goal was not to make a generic visualisation of any DEMKit configuration, just the specific situation with six houses. Since the DEMKit settings are also part of our deliverables, and the network configuration is set there, we judge we can assume that in the rest of the system the network is as we expect.

Pushing the simulation state

For getting the simulation state from the simulation component to the visualisation component we had to choose between two options: pulling the state from the simulation through an endpoint or pushing the state to all clients. We went with the latter option. This option is focused more on the simulation being the main source of truth for all connected clients. This way the simulation just pushes its state when it thinks it is necessary, i.e. when a change like the time

advancing by one or more tick happens. When using the former option all clients would independently query DEMKit, often sending requests that are not necessary because no change occurred. This is a waste of resources.

Controlling the simulation time

For the problem of controlling the simulator we thought of two approaches: one would implement a /pause and a /play endpoint, and the simulator would simply start running after a /play command, and stop after a /pause. The other implementation would be a /tick endpoint. Every time the simulation should advance by one tick, so one basic unit of time, when this endpoint is called. This allows total control of the simulation speed by the front-end, with the limitation of the amount of requests it can successfully send per second.

Have the Node.JS server in between

At the start of development we had the frontend make some calls to DEMKit directly, without the use of the Node.JS server. Since the calls to DEMKit came from a different domain than where the DEMKit API was running we ran into Cross-Origin Resource Sharing (CORS) issues. This solidified the idea that all calls should go through the Node.JS server, because then calls would be made from the same domain. The server also has a more uniform way of keeping track of client IDs by using the Socket.IO ID.

Dumb/Smart button

With our client we determined that houses should be able to be controlled in a 'smart' or 'dumb' manner. Specifically this means adding special controllers to the devices in DEMKit. We had to choose between making all houses smart with one button, or have the user choose per house. We decided to go with the per house choice, so there is a button with a brain icon next to every house that lights up when clicked to signify that the house is now controlled in a smart manner. We chose this to allow users to see what happens when only a part of the street has adopted the smart grid paradigm. This can also show how much it can help if only one house in the street is smart.

Testing

Unit tests

We've written python unit tests to check whether the add- and remove-methods function for each entity in DEMKit work as intended. These tests are meant to determine whether these methods properly add and remove the entities to the correct list, as described in the Dynamic Model Updating section. The result of the tests was that these methods work as intended. In checking for other methods to be tested in this way, we determined that that wouldn't be useful. Most of the code we added to DEMKit is written to make DEMKit function as part of a network. Thus, it is not intuitive to write tests for only that part of the system. Even for the add- and remove- methods we did test this way, it was a challenge to isolate them from the rest of the system. Therefore, no unit tests have been written to test other methods.

Selenium tests

Selenium is a frontend testing automation tool. With Selenium we can automate user tests. Selenium allows us to record actions and play them back so we can automatically test certain actions in the system without actually inputting anything ourselves. This could also be used in a demo environment if you want to restrict users from interacting with the demo but have activity happening on the screen. Selenium works as a browser plug-in so it is easy to install and use with the web-based visualiser.

User testing

Testing was done on computers by us, as well as on the touchpad by us and by our supervisor. Through these tests, we discovered how the information displayed by our visualisation tool was lacking. Thus we developed different ways to show this information to the user, for example by using charge bars, greying out the screen when the simulation is paused, finding out how to display device settings, and putting a smart button on each house.

Issues and problems that arose were:

- The setting for the electric car couldn't be closed
- It was possible to drag a device under a setting window
- The display was optimized for a touchpad, so the settings windows in the top half are upside down on a computer screen
- When multiple users tried to use the system we noticed that Chrome, with its zoom-in feature, wasn't as optimal as Mozilla Firefox
- We also optimized the time it takes for devices to disappear when placed outside the house
- Fine-tuning the running speeds for the fast setting

Set-up

For the setup we would guide you to the added install guide of our system. This is a 7 page instruction manual which in detail explains how to install all the requirements but also how to run our system. We also included the instructions to setup DEMKit and the solid code for which we know it works. This is all provided in the extra zip file.

Evaluation

Planning

The planning was set once the group was formed and we were able to keep up with how it was organized. We did overestimate, as well as underestimate, certain tasks but in the end it essentially all levelled out. Based on what parts took longer and what parts did not take as long, we decided which additional features we were going to implement in the time we set out for those. We set out to meet with our client every week to discuss the progress of our project and get feedback. No particular management tool was used for the duration of the project.

Responsibilities

At the beginning of the project, we set to divide the project into different components and tasks. As the group was somewhat diverse we were able to distribute the tasks according to interest. The tasks in question were divided into the following components: DEMKit, web-based visualiser and other (presentations, design report, poster, ethics report, project proposal).

- Egbert: Web-based Visualiser: canvas drawing, user tests, docker integration, settings menu.
- Simona: Web-based Visualiser: UI elements and drawing, user tests, other: ethics report, poster
- Koen: Web-based Visualiser: making devices smart/dumb, DEMkit: dynamically adding devices to the simulation, making devices smart/dumb, dynamically removing devices from the simulation, dynamically editing devices in the simulation, other: poster
- Pieter: Web-based Visualiser: General structure and fixes, drawing of devices, network nodes cables and current particles, general UI Buttons, Client side DEMKit data management DEMKit: State-pusher, some endpoints in the DesignApi, general bug fixes
- Thomas: DEMKit: unit tests, Web-based Visualiser: user tests, other: presentations, structure and spell check reports

The design report and the project proposal are aspects of the project everyone contributed to so it is to be seen as a group effort.

Team evaluation

We as a group have never worked together before. Due to a different need of a work environment, there were some issues at the beginning with communication. The additional load of some of the team members doing multiple modules also resulted in problems of availability

and contact. To solve these issues we met with our supervisor and discussed the possible solutions. In the end, this resulted in more project meetings than anticipated at the beginning of the module, but in retrospect, it was for the best. Following that meeting, no other problems have arisen in the collaboration.

Final result

At the start, the project was envisioned to connect Tactile Triana and DEMKit. As stated in the design report already we decided to recreate Tactile Triana and connect the new system to DEMKit. Our smart grid demonstrator resulted in a difference of the missing detail graph screen from Tactile Triana. This was done as Grafana already has this functionality and was deemed by the client as less important. There are no unnecessary functionalities meaning the feedback received during the meeting sessions with the client were taken fully into account when making the system. At the end of the module the project resulted in a complete system covering all the must requirements set by the client.

Conclusion

There are many ways future developers can extend this product. A start can be made by looking at the features we mentioned that we would have liked to add, but were not feasible within the time frame of this project, such as a way to add more houses to the grid and smart-phone support for the application. The functionality to send information from DEMKit to multiple clients already exists, but it still needs to be thoroughly tested, since we decided to focus on one client. Once this has been proven to work, an additional visualiser can be created for smartphones, probably showing just one house. Then, multiple users can connect to DEMKit, with each person having control of one house on their phone, while the touchpad is showing the entire grid of the neighborhood.

Sources

[1]: Naur, P., Programming as Theory Building, <http://pages.cs.wisc.edu/~remzi/Naur.pdf>